

SKILL WEEK

Task 1: Combined Redirection

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

To combine standard output and standard error redirection into a single file using file descriptors.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("combined.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd < 0) {
        perror("combined.txt");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        dup2(fd, STDOUT_FILENO);
        dup2(fd, STDERR_FILENO);

        close(fd);

        printf("This is standard output.\n");
        fprintf(stderr, "This is standard error.\n");

        exit(0);
    }
}
```

```

    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("Combined redirection completed successfully.\n");

    return 0;
}

```

Input

No keyboard input is required.

The program generates:

Standard Output

Standard Error

Output

```

amrutha@blue:~$ mkdir -p ~/SKILL_Week8
amrutha@blue:~$ cd ~/SKILL_Week8
amrutha@blue:~/SKILL_Week8$ nano combined.c
amrutha@blue:~/SKILL_Week8$ gcc combined.c -o combined
amrutha@blue:~/SKILL_Week8$ ./combined
Combined redirection completed successfully.
amrutha@blue:~/SKILL_Week8$ cat combined.txt
This is standard error.
This is standard output.
amrutha@blue:~/SKILL_Week8$ cat combined.txt
This is standard error.
This is standard output.

```

Observation

Both stdout and stderr were redirected to the same file using duplicated file descriptors. The output streams were successfully merged into combined.txt.

Result

Combined output and error redirection were successfully implemented and verified.

Task 2: Complex Command Execution

Task

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

Aim

To parse a command containing pipes and redirection symbols and generate an execution plan showing the order of commands and redirections.

Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[500];
    char *command;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\n--- Execution Plan ---\n");

    command = strtok(input, "|");

    int count = 1;

    while (command != NULL) {
        printf("Command %d: %s\n", count, command);

        if (strchr(command, '>') != NULL)
            printf("  Output redirection detected.\n");

        if (strchr(command, '<') != NULL)
            printf("  Input redirection detected.\n");

        if (strstr(command, "2>") != NULL)
            printf("  Error redirection detected.\n");

        command = strtok(NULL, "|");
        count++;
    }

    if (count > 2)
        printf("Pipeline detected.\n");
}
```

```
    printf("Execution plan generated successfully.\n");  
    return 0;  
}
```

Input

cat input.txt | grep Hello > output.txt

cat input.txt | grep Hello | wc -l > count.txt

Output

```
amrutha@blue:~/SKILL_Week8$ nano complex.c  
amrutha@blue:~/SKILL_Week8$ gcc complex.c -o complex  
amrutha@blue:~/SKILL_Week8$ ./complex  
myShell> cat input.txt | grep Hello > output.txt  
  
--- Execution Plan ---  
Command 1: cat input.txt  
Command 2:  grep Hello > output.txt  
    Output redirection detected.  
Pipeline detected.  
Execution plan generated successfully.  
amrutha@blue:~/SKILL_Week8$ ./complex  
myShell> cat input.txt | grep Hello | wc -l > count.txt  
  
--- Execution Plan ---  
Command 1: cat input.txt  
Command 2:  grep Hello  
Command 3:  wc -l > count.txt  
    Output redirection detected.  
Pipeline detected.  
Execution plan generated successfully.
```

Observation

The input command was divided into individual pipeline commands. Redirection operators were detected and the execution structure was displayed in the correct command order.

Result

Complex command parsing and execution-plan generation were successfully implemented.

Task 3: Background Jobs

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

To detect a background command using &, create a non-blocking child process, and return the shell prompt immediately.

Program

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("myShell> ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    int len = strlen(command);

    if (len > 0 && command[len - 1] == '&') {

        command[len - 1] = '\0';

        while (len > 1 && command[len - 2] == ' ') {
            command[len - 2] = '\0';
            len--;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            return 1;
        }

        if (pid == 0) {
            execlp(command, command, NULL);

            perror("exec");
            exit(1);
        }
    }
}
```

```

    }

    printf("[1] Background process started with PID %d\n", pid);
    printf("myShell> Prompt returned immediately.\n");
}
else {
    printf("Foreground command detected.\n");
}

return 0;
}

```

Terminal Commands

cd ~/SKILL_Week8

nano background.c

Compile:

gcc background.c -o background

Run:

./background

Input

sleep 5 &

Output

```

amrutha@blue:~/SKILL_Week8$ nano background.c
amrutha@blue:~/SKILL_Week8$ gcc background.c -o background
amrutha@blue:~/SKILL_Week8$ ./background
myShell> sleep 5 &
[1] Background process started with PID 2981
myShell> Prompt returned immediately.

```

myShell> sleep 5 &

[1] Background process started with PID 2451

myShell> Prompt returned immediately.

The PID will be different on your system.

Another Test

```

amrutha@blue:~/SKILL_Week8$ ./background
myShell> sleep 10 &
[1] Background process started with PID 3017
myShell> Prompt returned immediately.

```

Input:

sleep 10 &

Output:

myShell> sleep 10 &

[1] Background process started with PID 2470

myShell> Prompt returned immediately.

Observation

The & symbol was detected as a background-job indicator. A child process was created using fork(), and the parent did not wait for the child, allowing the prompt to return immediately.

Result

Background process detection and non-blocking execution were successfully implemented.

Task 4: Job Table

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

To maintain a job table containing process IDs and job states, update their status, and remove completed jobs.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 5

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

int main() {
    struct Job jobs[MAX_JOBS];
    int job_count = 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(2);
        exit(0);
    }

    jobs[job_count].id = 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state,
             sizeof(jobs[job_count].state),
             "Running");

    job_count++;
}
```



```

printf("--- Job Table ---\n");

for (int i = 0; i < job_count; i++) {
    printf("[%d] PID: %d State: %s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

waitpid(pid, NULL, 0);

for (int i = 0; i < job_count; i++) {
    if (jobs[i].pid == pid) {
        snprintf(jobs[i].state,
                 sizeof(jobs[i].state),
                 "Completed");
    }
}

printf("\n--- Updated Job Table ---\n");

for (int i = 0; i < job_count; i++) {
    printf("[%d] PID: %d State: %s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

job_count--;

printf("\nCompleted job removed.\n");
printf("Remaining jobs: %d\n", job_count);

return 0;
}

```

Input

No keyboard input is required.

The program automatically creates a child process and stores it in the job table.

Output

```
amrutha@blue:~/SKILL_Week8$ nano jobtable.c
amrutha@blue:~/SKILL_Week8$ gcc jobtable.c -o jobtable
amrutha@blue:~/SKILL_Week8$ ./jobtable
--- Job Table ---
[1] PID: 3067 State: Running

--- Updated Job Table ---
[1] PID: 3067 State: Completed

Completed job removed.
Remaining jobs: 0
```

Observation

A job table was created to store the job ID, process ID, and current state. The process state was updated from Running to Completed, after which the completed job was removed.

Result

Job table creation, process tracking, state updating, and removal of completed jobs were successfully implemented.